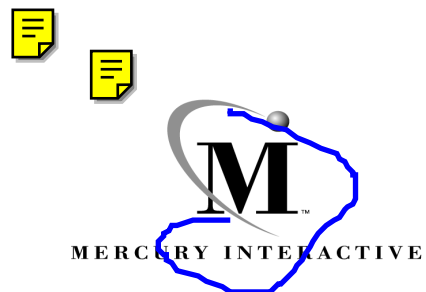


Introduction to TestSuite 7.0

WinRunner - Student Workbook



EXERCISES: Recording and Playback

Create the Test Framework

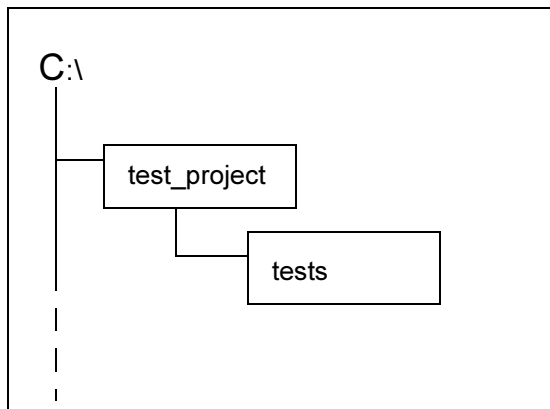


Why should you create a real test framework on a shared, network drive?



NOTE: For training purposes, you will create a test framework on your local drive. The training will refer to this framework when saving lab exercises.

1. Create a test framework on your local file system using the folder names shown in the following diagram:



Later in the training, you will add more to the framework.

Exercise A: Record and Play Back

Record and play back a test using the Flight Reservation application version 1A as the application under test (AUT).

Setup

1. Invoke WinRunner.

From the Windows Start menu, select Programs > WinRunner > WinRunner, or double-click the WinRunner desktop icon.

2. Close the Add-In Manager.

As WinRunner opens, the Add-in Manager dialog appears. All Add-In options should be unselected.

Click OK in the Add-In Manager.

3. If the Welcome to WinRunner dialog appears, uncheck the *Show at startup* check box, then click the *New Test* icon.



4. If the Welcome to WinRunner dialog did not appear, start a new test.

Click the *New* icon on WinRunner's toolbar, or Select File > New.

A blank test script window opens.

5. Invoke version 1A of the Flight Reservation application.

From the Windows Start menu, select Programs > WinRunner > Sample Applications > Flight 1A, or double-click the desktop icon labeled *Flight1A*.

6. Log on to the Flight Reservation application.

Type your name in the *Agent Name* field.

Type mercury in the *Password* field and click OK. The Mercury Tours logo appears before the Flight Reservation window is displayed.

7. Position the windows so that the Flight Reservation window and the WinRunner window are both visible without overlapping.

Record Opening an Order



1. Start Recording in WinRunner.

Press the *Record* toolbar button, or select Create > Record - Context Sensitive.

2. Open order number 1 in the Flight application.

Select File > Open Order in the Flight Reservation window.

Check the *Order No.* check box in the Open Order window.

Type 1 in the text box, and press OK.

3. Open the Graph window.

Select Analysis > Graphs in the Flight Reservation window.

4. Close the Graph window.



5. Stop recording in WinRunner.

Press the *Stop* toolbar icon, or select Create > Stop Recording.

6. Save the test as: C:\test_project\tests\lab1a

Select File > Save As. The Save Test dialog box opens.

Navigate to the folder where you want to save the file

(C:\test_project\tests).

Type lab1a in the *File name* edit box.

Press the *Save* button when you are ready to save.

Play Back and View the Results



1. Run the test from the top of the script.

Press the *Run from Top* button on the toolbar, or select Run > Run from Top.

2. Accept the suggested *Test Run Name*.

Press OK in the Run Test dialog box.

3. Examine the Test Results.

The WinRunner Test Results window appears at the end of the run.

The *Test Result* line indicates “OK” which means that the test passed.

4. Close the WinRunner Test Results window.

Select File > Exit.

Exercise B: Set Initial Conditions

In this exercise you will learn the importance of the initial conditions of the AUT when recording a test.

Setup



1. Start a new WinRunner test.

Click the *New* toolbar icon, or select File > New.

2. Start a new order in the Flight application.

Select File > New Order.

Record Creating an Order



1. Start recording.

Press the *Record* button on the WinRunner toolbar, or select Create > Record - Context Sensitive.

2. Enter the date of flight.

Click in the *Date of Flight* edit field, and type the date 12/12/03.

3. Select the flight route.

Select “Los Angeles” from the *Fly From* drop-down list.

Select “Denver” from the *Fly To* drop-down list.

4. Select a flight number.

Press the *Flights* button.

Select a flight from the Flights Table and press OK.

5. Enter the customer name.

In the Flight Application, click in the *Name* field and type your name.

6. Finish the data entry.

Check the *First* radio button under *Class*.
Type 2 in the *Tickets* field.



During recording, it is recommended to click on edit fields before typing in any data and to avoid using the tab key to move from one field to another. WinRunner records tab key presses as well as data entry. Avoiding tabs will result in cleaner, easier to read scripts.

7. Create the order.

Press the *Insert Order* button. After the order is inserted into the database, the message “Insert Done...” appears in the progress bar.



8. Stop recording.

Press the *Stop* toolbar icon, or select Create > Stop Recording.

9. Save the test as: C:\test_project\tests\lab1b

Examine the Script



Describe the operation performed by each of the following statements:



Use the *Context Sensitive Help* icon  and click on any statement.

```
set_window ("Flight Reservation", 10);
```

```
edit_set ("Date of Flight:", "12/12/03");
```

```
list_select_item ("Fly From:", "Los Angeles");
```

```
button_press ("Flights...");
```

```
button_set ("First", ON);
```

Playback



1. Play back the test.

Press the *Run from Top* button on the toolbar, or select Run > Run from Top, and accept the suggested *Test Run Name*.



Did the test play back to completion? If not, why not?

2. Press the *Pause* button in the WinRunner Message box.

The test run paused at the line: `button_press ("Insert Order");`



Were the initial conditions of the application the same when you recorded the test and when you ran it? How can you correct the problem?

Set the Initial Conditions

Edit the test so that the initial condition of the application is the same each time you run the test. Do this by recording a File > New Order command at the beginning of the test.

1. Edit the test script.

In WinRunner, insert an empty line at the top of the script, before the first statement.

Place the cursor in the empty line.



2. Start recording.

Press the *Record* button, or select Create > Record-Context Sensitive.

3. Set the initial conditions for the AUT.

In the Flight Reservation window, select File > New Order.



4. Stop recording.

In WinRunner, press the *Stop Recording* button in the toolbar, or select Create > Stop Recording.

5. Dismiss the superfluous window in the AUT.

In the Flight Reservations message window, press the *Cancel* button.



NOTE: Ordinarily, this window will not appear, so you don't want to record it.



6. Play back the test twice.

Press the *Run from Top* toolbar icon.

Uncheck the *Display test results at end of run* check box. This allows you to watch the AUT to the end of the playback.

Press OK in the Run Test dialog box.

When the run has finished, run the test again.



Why is it a good idea to run the test twice?

7. Save the test.

Exercise C: Context Sensitive and Analog Recording Modes

In this exercise, you will combine Context Sensitive recording with Analog recording. You will learn when it is advantageous to use each recording mode.

Record Context Sensitive and Play Back



1. Start a new test in WinRunner.



2. Start recording.



3. Open order number 1 in the Flight application.

In the Flight Reservation window, press the *Open Order* icon or select File > Open Order.

Check the *Order No.* check box.

Type 1 in the edit box.

Press the *OK* button.

4. Open the Fax Order window.

Select File > Fax Order in the Flight Reservation window.

5. Enter the fax number.

Type 408 555 1212 in the *Fax Number* field.

6. Sign your first name in the *Agent Signature* area.

Move the cursor inside the *Agent Signature* box. The cursor changes shape to look like a pencil.

Hold down the left mouse button, and write your first name in the box with the cursor.

7. Send the fax.

Press the *Send* button.

Wait for the “Fax Sent” message to appear in the Progress Bar.



8. Stop recording.



9. Play back the script without displaying test results at the end.



Was the signature reproduced during playback?

Record Analog and Play Back

1. Put the cursor at the bottom of the script.

2. Press `ENTER` to insert a blank line above the cursor.

3. Select **Create > Record Analog**.

4. Repeat the steps to fax an order from the previous section (**Record Context Sensitive and Play Back**).

Open the Fax Order window.

Enter the fax number.

Sign your first name in the *Agent Signature* area.

Send the fax.



5. Stop recording.

6. Put the run arrow at the first line of the analog recording.

Click in the column to the left of the first line that begins: `move_locator_track`.



7. Select **Run > Run from Arrow without displaying test results at the end**.



Was the signature reproduced during playback?



Can you tell the difference between the Context Sensitive statements and the Analog statements? Which are more intuitive?



In what testing situation would it be appropriate to use analog recording mode rather than context sensitive?

Combine Context Sensitive and Analog Recording



1. Start a new test in WinRunner.



2. Start recording in context sensitive mode.



3. Open order number 1 in the Flight application.

4. Open the Fax Order window.

5. Enter the fax number.

6. Move the Fax Order window to a new position on the screen.

7. Switch to Analog record mode in WinRunner using the softkey.

Press the F2 key. The status line displays “Recording - Analog Mode.”

8. Write your first name in the *Agent Signature* box in the Flight application.

9. Switch back to Context Sensitive record mode in WinRunner with the softkey.

Press the F2 key again. The status line displays “Recording - Context Sensitive Mode.”



Why is it good practice to use the softkey to switch recording modes rather than the menu?

10. Send the fax in the Flight application.



11. Stop recording in WinRunner.



Before switching to Analog mode and signing your name, you were asked to move the window to a new position. Why is it important to move a window to a specific location before recording in Analog mode?



12. Play back the script without displaying test results at the end.



Was the signature reproduced during playback?



Was the script easier to read than the one recorded in analog mode?

13. Save the test as: `C:\test_project\tests\lab1c`

Exercise D: The GUI Map — An Introduction

1. **Open the GUI Map Editor.**
Select Tools > GUI Map Editor.
2. **Unload the GUI Map File.**
Select File > Close All.
3. **Close the GUI Map Editor.**
Select File > Exit.



4. Play back the script.



Did the script play back? If not, why not?



A WinRunner script cannot run without a loaded GUI map.
Press *Cancel* to close the Run Wizard dialog box.

5. Open the GUI Map Editor.

6. Load the GUI Map.

Select File > Open. The Open GUI File dialog box opens.

Navigate to C:\merc_trn70\trn_init and select *flight70.gui*.

Press the *Open* button.

7. Close the GUI Map Editor.

8. Play back the script.



Did the script play back?

9. Close all open tests.

Select Window > Close All.

Do not save the “noname” test.

10. Close the Flight application.

Select File > Exit.



What is the general purpose of the GUI Map?

Review: Recording and Playback

Test your understanding of this lesson by answering the following questions:



1. When would you record a test in Analog mode instead of Context Sensitive mode?



2. Assume you are recording in Analog mode and you want to stop recording. Which method would you use: softkey, menu command, or toolbar? Why?



3. Think about **your** application. Would use Context Sensitive, Analog or both methods of recording?

EXERCISES: Add Synchronization Points

In this lesson you will learn how to insert synchronization points in your test. These points ensure that during playback WinRunner will wait for a certain event before continuing execution.

Exercise A: No Synchronization

In this exercise you will record a test in which you login to the Flight Reservation application. You will see what happens during playback when the test is not synchronized with the AUT.

Setup



1. Start a new WinRunner test.
2. Invoke version 1A of the Flight application and log in.

Record Creating a New Order



1. Start recording.



2. Set the initial condition of the test.
In the Flight Reservation window, click the *New Order* toolbar icon.
3. Type 12/12/03 in the *Date of Flight* field.
4. Select the flight route and number.
Select any *Fly From* city and any *Fly To* city.
Press the *Flights* button and select any flight from the list.
5. Type your name in the *Name* field.

6. Finish the data entry.

Type 3 in the *Tickets* field.

7. Create the order.

Press the *Insert Order* button.

After the order is inserted into the database, the message “Insert Done...” appears in the progress bar at the top of the window.

**8. Delete the order.**

Press the *Delete Order* button.

Press the *Yes* button in the message window to confirm the delete.

**9. Stop recording.****10. Save the test as: C:\test_project\tests\lab2a****Play Back the Recording****1. Play back the test.**

Did a WinRunner message appear? If yes, what is the message?

2. Close the WinRunner Message dialog box.

Press *Pause*.

Exercise B: Synchronize Using `obj_wait_bitmap`

Use the `obj_wait_bitmap` command to synchronize a test with the appearance of text in the progress bar of the Flight application.

Add Synchronization

1. **Insert a blank line after the `button_press("Insert Order")` statement in the script and put the cursor on it.**
2. **Synchronize the test to wait for the “Insert Done...” bitmap.**
 Select Create > Synchronization Point > For Object / Window Bitmap. WinRunner minimizes and the cursor changes shape to resemble a pointing hand. Using the hand pointer, click on the progress bar message “Insert Done...” WinRunner restores and inserts an `obj_wait_bitmap` statement into the script.
3. **Change the time parameter to 20 seconds.**
 In the statement `obj_wait_bitmap("StatusBar95", "Img1", 1);` the last parameter, 1, is the time in seconds that WinRunner will wait before capturing the bitmap from the AUT to compare with its stored image. Change the 1 to 20 so that WinRunner will wait longer.



4. **Save the test as:** `C:\test_project\tests\lab2b`

Play Back and View the Results



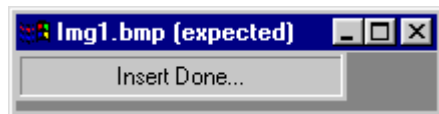
1. **Play back the test.**



Did the test play back successfully this time?

**2. Examine the Test Results.**

Click on the line showing “wait for bitmap” in the *Event* column to highlight it. Click the *Display* button, or double-click on the line to see the bitmap that was captured.



In the Test Results window, bitmap synchronization lines that passed are displayed in green. Failed synchronization lines are displayed in red.

3. Close the bitmap window.**4. Close the WinRunner Test Results window.**

Why is an `obj_wait_bitmap` statement potentially unreliable?



Can you think of a more reliable method of synchronization?

Exercise C: Synchronize Using *obj_wait_info*

In Exercise B, you used the `obj_wait_bitmap` function to synchronize the test with the progress bar message. While this is a valid method, it relies on screen resolution, color, etc.

In this exercise you will synchronize the test with the same progress bar message, using the `obj_wait_info` function, which is a context sensitive function. This function will wait for the label attribute of the progress bar GUI object to reach the value “Insert Done...”



NOTE: *Attribute* and *property* are used interchangeably in this training. WinRunner uses attribute in some dialog boxes and property in others to refer to the same things.

Setup

- 1. Add a breakpoint at the `obj_wait_bitmap` statement.**
Right-click in the column to the left of the line containing the `obj_wait_bitmap` statement. The WinRunner breakpoint symbol—a red hand—appears on the line.
- 2. Play back the script.**
Playback stops at the breakpoint.
Wait for the order to be inserted.
- 3. Change the `obj_wait_bitmap` line to a comment.**
Type # at the beginning of the line containing the `obj_wait_bitmap` function.
WinRunner will not execute a line beginning with the comment symbol [#].
- 4. Remove the breakpoint.**
Right-click on the breakpoint symbol.
- 5. Insert a blank line immediately after the comment line and put the cursor on it.**

Synchronize on "Insert Done"

1. **Select Create > Synchronization Point > For Object/Window Property.**
2. **Insert a statement to wait for the "Insert Done" message.**
Using the pointing hand cursor, click on the progress bar containing the text "Insert Done..." The Wait For Object dialog box appears.
Select "label" from the *attribute* drop-down list.
Note that the *exp_value* (expected value) argument automatically changes to "Insert Done..."



Why did you select the label attribute from the attribute list?

3. **Assign a timeout value of 20 seconds.**
Type 20 in the *timeout* field.



Will WinRunner wait longer than 20 seconds if it needs to?

4. **Insert the function into the test script.**
Press the *Paste* button.
5. **Save the test as:** C:\test_project\tests\lab2c

Play Back and View the Results



1. **Play back the test from the top.**



2. **Examine the test results.**



What was the result of the modified test?



NOTE: There is no line in the Test Results window for the `obj_wait_info` statement as there was for the `obj_wait_bitmap` statement.

3. **Close the WinRunner Test Results window.**
4. **Close all tests.**
Select Window > Close All.
5. **Close the Flight Application.**

Review: Add Synchronization Points

Test your understanding of this lesson by answering the following questions:



1. In which cases do you need to insert synchronization points in your script?



2. Which method would you use to synchronize the following:

(a) Window: _____

(b) Bitmap image: _____

(c) Status bar message: _____



3. What are the differences between `obj_wait_bitmap` and `obj_wait_info`?



4. Think about **your** application. Where are synchronization points needed? Which method you would use?

EXERCISES: Bitmap Checkpoints

In this exercise you will learn how to use bitmap checkpoints in your test and view the results. You will create a test that checks the signature in the Fax Order window.

Fax Order No. 7

Fax

Name:	Order:	Flight:	Date:
Jane Doe	7	3291	07/17/99
From:	Departure:	To:	Arrival:
San Francisco	08:00 AM	Los Angeles	08:45 AM
Class:	# Tickets:	Ticket Price:	Total:
First	9	301.50	2713.50
Fax Number:	Agent Signature:		
(408)555-1212			
☒ Send Signature with order			

Preview Fax Send Cancel Clear Signature

You will check that the *Clear Signature* button removes the signature from the window. To perform this check you will capture an object image of the *Agent Signature* area.

Setup



1. Start a new test in the WinRunner window.
2. Invoke Flight application 1A and log in.

Record Faxing an Order



1. **Start recording.**
2. **Open order number 7 in the Flight Reservation window.**
3. **Open the Fax Order window.**
Select File > Fax Order.
4. **Enter the fax number 4085551212.**
5. **Set the screen position for the Fax Order window.**
Move the Fax Order window to a different position on the screen.
6. **Switch to Analog mode using the softkey.**
Press the F2 key. The status line displays “Recording-Analog Mode.”
7. **Sign your name in the Agent Signature box.**
Using the mouse, sign your name in the *Agent Signature* box.
8. **Switch back to Context Sensitive mode with the softkey.**
Press the F2 key again. The status line displays “Recording-Context Sensitive Mode.”
9. **Clear your signature.**
Press the *Clear Signature* button.
-  10. **Insert a Bitmap Checkpoint.**
Select Create > Bitmap Checkpoint > For Object / Window.
Click inside the *Agent Signature* area.
WinRunner inserts an `obj_check_bitmap` statement into the script.
11. **Send the fax order.**
Press the *Send* button in the Fax Order window.



12. Synchronize the test with the Fax Sent message using an `obj_wait_info` function with a 20 second timeout.

Refer to Exercise C of the *Add Synchronization Points* lesson if you need help with this step.



13. Stop recording.



14. Save the test as: `C:\test_project\tests\lab4a`

View the Expected Results



1. Open the Test Results.



2. Open the image.
Double-click the capture bitmap line.



What does the image show?

3. Close the Test Results window.

4. Close the Flight application.

Play Back on Flight1B

1. Invoke Flight1B and log in.
Wait for the Flight Reservation window to appear.



2. Run the test in Verify mode.
3. Change the Test Run Name to `Regression3`.
4. Press *Continue* if any WinRunner messages appear.

View the Failed Results

1. **View the expected, actual and difference images of the bitmap checkpoint that failed.**

Double-click on the bitmap checkpoint line marked in red.



The type of image (expected, actual, difference) appears in the title bar of each image window. You might have to resize the image windows to be larger before you can tell which image is which.



What was the reason for failure?

2. **Close the WinRunner Test Results window.**
3. **Close all open tests.**
4. **Close the Flight Reservation application.**

Review: Bitmap Checkpoints

Test your understanding of this lesson by answering the following questions:



1. Assume that you created a test with a bitmap checkpoint on one machine, and you run the test on another machine with a different resolution or color setup. What problem would you encounter? How can you solve it?



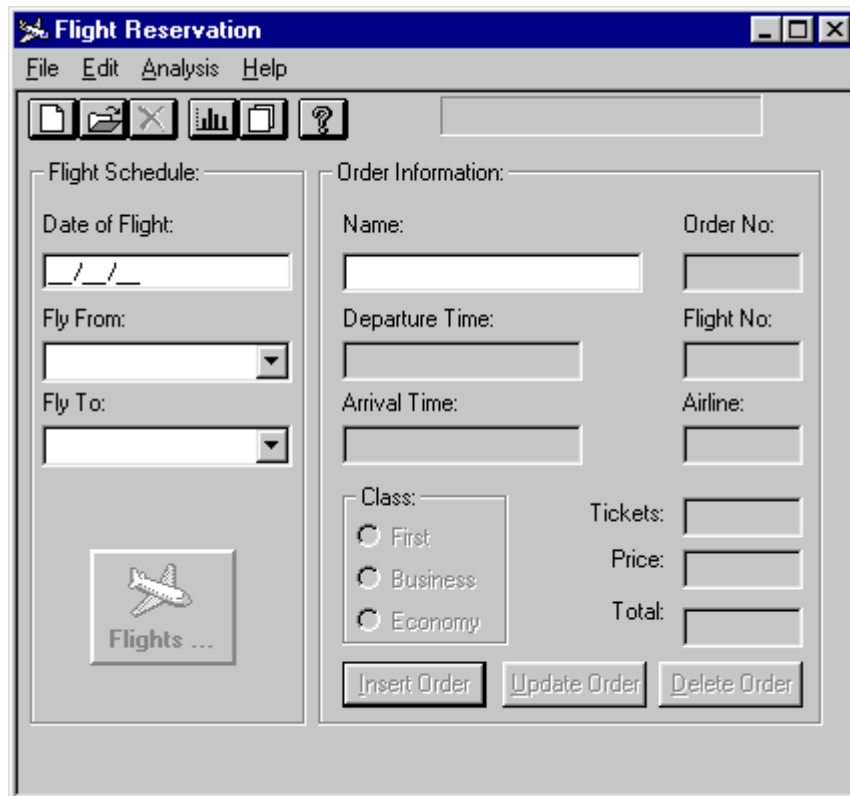
2. Are there areas in **your** application which you would test using a bitmap checkpoint?

EXERCISES: GUI Checkpoints

In these exercises you will create a GUI checkpoint in a test, reuse a GUI checklist, view GUI check results, and determine when to use Debug and Verify playback modes. You will create a test on version 1A of the Flight application and run it on version 1B in order to perform regression testing.

Exercise A: Check Multiple Objects Using a Checklist

In this exercise you will create a test that checks the initial conditions of the Flight Reservation window and performs a “sanity” check on the order insertion process. You will learn how to save time by reusing a GUI checklist.



The screenshot shows the 'Flight Reservation' application window. It has a menu bar with 'File', 'Edit', 'Analysis', and 'Help'. Below the menu bar is a toolbar with icons for file operations and help. The main area is divided into two panels. The left panel, titled 'Flight Schedule:', contains fields for 'Date of Flight:' (with a date picker), 'Fly From:' (a dropdown menu), and 'Fly To:' (a dropdown menu). Below these is a button labeled 'Flights ...' with an airplane icon. The right panel, titled 'Order Information:', contains fields for 'Name:', 'Order No:', 'Departure Time:', 'Flight No:', 'Arrival Time:', and 'Airline:'. Below these are radio buttons for 'Class:' with options 'First', 'Business', and 'Economy'. To the right of the class options are fields for 'Tickets:', 'Price:', and 'Total:'. At the bottom of the right panel are three buttons: 'Insert Order', 'Update Order', and 'Delete Order'.

Setup



1. Start a new test in the WinRunner window.
2. Login to the Flight application version 1A.



3. Start recording.



4. Set the initial conditions for the test.

Insert a GUI Checklist



1. Open the Create GUI Checkpoint dialog box.
Select Create > GUI Checkpoint > For Multiple Objects.
2. Begin checking the GUI objects.
Press the *Add* button in the Create GUI Checkpoint dialog box.
3. Check that the edit fields are empty of content.
Using the hand pointer, click inside each of the following edit fields:

EDIT FIELD
Date of Flight
Name
Tickets

4. Check the content of the list fields and that the selection is empty.
Click inside each the following list fields:

LIST FIELD
Fly From
Fly To

5. Check that the buttons are disabled.

Click on each of the following buttons:

PUSH BUTTON
Flights
Insert Order
Update Order
Delete Order

6. Press the right mouse button to quit adding checks and to restore the Create GUI Checkpoint dialog box.



What is the default check for each type of object?

Edit field? _____

List field? _____

Push button? _____

7. Save the GUI Checkpoint information.

Press the *OK* button.

Wait for the `win_check_gui` statement to be inserted into the test script.



How can you test the checkpoint for both pass and fail conditions?

Reuse the Checklist

1. Complete the data entry for a new flight reservation using data from the following table:

FIELD	DATA
Date of Flight	12/12/03
Fly From	Sydney
Fly To	Zurich
Flight	<pick any from the list>
Name	<your name>
Class	First
Tickets	4

2. Insert the order in the Flight application.
3. Synchronize on the “Insert Done” text in the progress bar.
Refer to the exercises in the *Synchronization* lesson if you need help with this.



4. Insert a GUI Checkpoint using the checklist you already created.
In WinRunner, select Create > GUI Checkpoint > For Multiple Objects.
Press the *Open* button in the Create GUI Checkpoint dialog box.
Select “list1.ckl” from the Open Checklist dialog box and press *OK*.
Press *OK* in the Create GUI Checkpoint dialog box.
Wait for the `win_check_gui` statement to be entered into the test script.



What is the difference between the first and second `win_check_gui` statements?



NOTE: The technique used in this step uses the existing checklist objects but recaptures the expected values based on the current state of the application.

5. Close the Flight application.
Select File > Exit.



6. Stop recording.



7. Save the test as: C:\test_project\tests\lab5b

Play Back on Flight1A

1. Login to the Flight application 1A.



2. Play back the test in Debug mode.



3. Open the Test Results.

Examine the results of each *end GUI checkpoint* event in the test results.



Did both checkpoints pass?

4. Close all WinRunner results windows.

Play Back on Flight1B and Check Results

Using version 1B of the application, run the Lab5b test created with version 1A.

1. Invoke Flight application 1B and log in.



2. Play back the test in Verify mode.

3. Change the *Test Run Name* to Regression2.

4. Press *Continue* if any WinRunner messages appear.



Did the test pass? If not, why not?

5. Close all WinRunner results windows, then close all tests.

Review: GUI Checkpoints

Test your understanding of this lesson by answering the following questions:



1. In this exercise you did not check any menu items. What would you check in a menu item?



2. Which areas in **your** application require GUI Checkpoints?

EXERCISES: Verifying Text

In this exercise you will create a test that checks the Flight application's version number which appears in the Help > About window.



Exercise A: Read Text from a Bitmap Area

Setup



1. Open a new test in the WinRunner window.
2. Invoke Flight1A and log in.

Record



1. Start recording.
2. Open the About window in the Flight application.
Select Help > About.



3. Read the version number text.

Select Create > Get Text > From Screen Area. The pointer becomes crosshairs. Move the cursor a little outside the text area you want to capture (“Version 1.0”). Press and hold down the left mouse button. Drag the cursor toward the text area so that a rectangle appears. Move the cursor so that the text you want to capture (and no other text) is enclosed by the rectangle, then release the mouse button. Press the right mouse button to complete the operation.



What statement was generated by WinRunner?



Was a comment generated? What was it?

4. Close the About window.

Press the *OK* button.



5. Stop recording.



When reading text, always notice what WinRunner writes in the comment. When WinRunner cannot read the text you marked, the comment will say: “No text was found.” In that case, you will first need to teach WinRunner the font of the text.

6. Verify the text.

Create a condition in the test script, that verifies the appearance of the text “Version 1.0” and sends a pass (“Correct version number”) or fail (“Wrong version number”) status to the test results report.

Insert an empty line after the `win_get_text` function, and type a test segment using the following elements:

- (a) an `if` statement
- (b) a variable that stores the expected result
- (c) the not equal to (`!=`) operator
- (d) the `tl_step` function

If you need help with this step, refer to the slides in this lesson.



7. **Save the test as:** `C:\test_project\tests\lab6a`

Play Back on Flight1B

Play back your test on version 1B of the Flight application. During the run, WinRunner compares the text stored in the variable *text* to the actual text in the application. If a mismatch is discovered, a message to that effect is sent to the test results report.

1. **Close the Flight application.**
2. **Invoke Flight1B and log in.**



3. **Play back the test in Verify mode.**
4. **Change the *Test Run Name* to** `Regression4`.

View the Test Results

1. **View the test results.**



Did Flight1B pass the test? If not, why?

2. **Close the WinRunner Test Results window.**
3. **Close all the tests that are currently open.**
4. **Close Flight application 1B.**

Exercise B: Learn a Font

In some cases WinRunner must first be taught the font used by the AUT before it can read any text. Each specific font size and style must be learned separately.

Setup



1. **Open a new test in the WinRunner window.**

2. **Open the Paint application.**

Run the Paint application that comes with Windows (look under \Program Files\Accessories). Depending on the version of Windows you are using, it might be called Paint, MSPaint, or Paintbrush. We will refer to it as the “Paint application” throughout this exercise.

3. **Resize and position the WinRunner and Paint application windows so that both are visible without overlapping.**



4. **Define the font in the Paint application window.**

Click the text tool in the Paint toolbox, then click in the drawing space. The Fonts dialog box should open. (If it does not, right-click inside the dotted-line rectangle and select “Text Toolbar.”)

Select the font “System.”

Select the size “10 pts.”

Select the style “Bold.”

Close the Fonts dialog box.

Enter and Read Text in the Paint Application

1. **Type your name in the Paint application window.**

Select the Text tool in the Paint application. The pointer changes into crosshairs.

Draw a rectangle. Hold down the left mouse button and drag the pointer so that it creates a rectangle large enough for you to type your name inside. Release the mouse button when finished. A cursor appears inside the rectangle.

Type your name inside the rectangle.

Click once outside the rectangle so that the dotted lines are no longer visible.



2. Read the Text.

In WinRunner, select Create > Get Text > From Screen Area. The WinRunner window is minimized.

Select the area by using the left mouse button to draw a rectangle around your name in the Paint application window.

Press the right mouse button to complete the operation.

3. Examine the script.

Notice that the text was not recognized. An `obj_get_text` statement was added to the script with the comment: `#No text was found`. This indicates that WinRunner could not recognize the font and therefore could not read the text.

Define a Font

In this section you will define a font and teach it to WinRunner.

1. Display the Fonts Expert window without overlapping the Paint window.

Select Tools > Fonts Expert in the WinRunner window.

Reduce the size of the Fonts Expert window, and position it so that it does not overlap the Paint application.

2. Open the Learn Font window.

Select Font > Learn in the Fonts Expert window.

3. Enter the name of the font.

Enter the name `System` in the *Font name* field.

4. Define the font.

Press the *Select Font* button.

Select the font “System.”

Select the font style “Bold.”

Select the font size “10.”

Press the *OK* button.

5. Learn the font definition.

Press the *Learn Font* button.

The System font is now being learned. When the process is complete, the *Existing Characters* box displays a list of all the characters learned, and the *Properties* frame displays the properties of the System font that was learned.

6. Close the Learn Font window.

Press the *Close* button.

Create a Font Group

Add the System font to a font group named “Tutorial.” You can add additional fonts to this group or create additional font groups, depending on your testing needs.

1. Open the Font Groups window.

Select *Font > Groups* in the Fonts Expert window.

2. Name the font group Tutorial in the Font Groups window.

Type `Tutorial` in the *Group Name* field.

3. Create the Tutorial group and include the System font.

Select “System” from the *Fonts in Library* list.

Press the *New* button.

A new group, Tutorial, is created which includes the font “System.”

4. Exit the Font Groups and the Fonts Expert dialog boxes.

Press the *Close* button to exit the Font Groups dialog.

Select *Font > Exit* to close the Fonts Expert dialog.

Set the Current Font Group

In order for WinRunner to recognize a font during test execution, you must set the appropriate font group. You do this using the `setvar` TSL statement.



1. **Open a new test in the WinRunner window.**

2. **Add a `setvar` statement to your test script.**

Type the following statement:

```
setvar ("fontgrp", "Tutorial");
```



3. **Execute the statement in Debug mode.**

Select Run > Run from Top.

Select *Use Debug mode* and press OK.

4. **Verify that Tutorial is the current font group.**

Select Settings > General Options. The Options dialog box opens.

Click the *Text Recognition* tab.

If the *Use Image Text Recognition mechanism* check box is not checked, then check it.

Verify that the *Font group* is set to Tutorial.

Press OK to close the dialog.

Enter Text in the Paint Application

1. **Open a New File in the Paint application window.**

Select File > New.

Do not save the changes to the “untitled” file.

2. **Type a sentence.**

Type the following sentence on a single line:

```
He is reading a book
```



NOTE: The color of the font is not important for reading text, as long as it is the same for all text. WinRunner will recognize any color.

3. **Click once outside the rectangle so that the dotted lines are no longer visible.**

Read the Paint Application Text



1. Read the text.

In the WinRunner window, place the cursor on an empty line, after the `setvar` statement.

Select Create > Get Text > From Screen Area.

Select the area by using the left mouse button to draw a rectangle around the sentence in the Paint application window.

Press the right mouse button to complete the operation.

2. The `obj_get_text` statement is inserted to your script.



Did the comment “He is reading a book” appear in the script?



Did WinRunner read the text?

3. Close all tests in WinRunner.

Do not save the “noname” tests.

4. Close the Paint application.

Do not save changes to the “untitled” file.

Review: Verifying Text

Test your understanding of this lesson by answering the following questions:



1. When would you read text using Get Text commands rather than using Context Sensitive functions?



2. Which areas in your application require text verification?

EXERCISES: Managing GUI Map Files

Exercise A: Learn the Windows/Objects of the AUT

Setup

1. **Create a directory in the automated test framework called:**

`C:\test_project\GUI_Map_files`



IMPORTANT: Do not use spaces in WinRunner paths! WinRunner will not find the GUI Map files if you put spaces in the path.

2. **Start version 1A of the Flight application and log on.**



3. **Set the proper initial conditions.**

4. **Open the GUI Map Editor in WinRunner.**

Select Tools > GUI Map Editor.

5. **View the GUI Map as individual files.**

Select View > GUI Files from the menu.



NOTE: A GUI Map may consist of more than one file. For convenience, WinRunner allows you to view the map as one file or as individual files.

6. **Check for open GUI Map files.**

Click on the *GUI File* list.



List the GUI Map files currently open or loaded.

-
7. **Close all GUI Map files.**

Select File > Close All.

Press Yes if a message appears asking if you want to clear the Temporary buffer.

Learn a Window Using the GUI Map Editor

1. Learn the Flight Reservation window.

Press the *Learn* button in the GUI Map Editor.

Click on the title bar of the Flight Reservation window with the pointing hand cursor.

Press *Yes* in the message box to learn all the objects in the window.



To which GUI Map file were all the learned objects added?



2. Locate the *Flights* button in the Temporary GUI Map file.

Press the *Find* button in the GUI Map Editor.

Using the pointing hand cursor, click the *Flights* button in the Flight application. WinRunner highlights the object in the GUI Map Editor.



What is the name of the *Flights* button in the GUI Map file?



List the attributes of the *Flights* button that WinRunner learned (look at the *Physical Description*).

3. Collapse the Flight Reservation tree in the GUI Map Editor.

Select View > Collapse Objects Tree.



You can also collapse the tree by double-clicking on the “Flight Reservation” node in the GUI Map Editor.

Learn Another Window

1. Open the Flights Table.

Type a valid date in the *Date Of Flight* field of the Flight application.

Select any *Fly From* city and any *Fly To* city.

Press the *Flights* button. The Flights Table opens.

2. Learn the Flights Table window.

Follow the same procedure you used to learn the Flight Reservation Window.

Close the Flights Table window in the Flight application when the GUI Map Editor has finished learning and you have finished examining the results.

3. Save the contents of the temporary GUI file into a new GUI Map file.

Select File > Save As.

Navigate to C:\test_project\GUI_Map_files.

Type lab11.gui in the *File name* field and press *Save*.



NOTE: Normally, you would continue learning the rest of the windows of the AUT.

Exercise B: Modifying GUI Files

Modify Object Names for Readable Scripts

In this section you will modify the names of some objects in the Flight Reservation window to make them more descriptive.

1. Expand the Flight Reservation tree in the GUI Map Editor.

Select the GUI Map File “lab11.gui” from the GUI File list.

Select View > Expand Objects Tree.



You can also expand the tree by double-clicking on the “Flight Reservation” node in the GUI Map Editor.

2. Find the GUI Map object called “Button_3” in the Flight application.

Click on Button_3 in the GUI Map Editor to highlight it.

Press the *Show* button in the GUI Map Editor.

In the Flight Reservation window, the highlighted object blinks.



Which Flight Reservation object is Button_3? What would be a more descriptive name for this button?

3. Modify the GUI Map name of the Button_3 object.

With the Button_3 object still highlighted in the GUI Map Editor, press the *Modify* button. The Modify dialog box opens.

In the *Logical Name* field replace Button_3 with Open Order - toolbar. Close the Modify dialog box by pressing OK.



What is the benefit of giving descriptive names to objects in the AUT?

4. Highlight the Open Order toolbar object in the GUI Map and press Show.

The *New Order* button in the Flight application's toolbar should blink.

5. Modify the remaining Button objects in the GUI Map.



What objects in the Flight Reservation window are associated with the following logical names in the temporary GUI file?

OBJECT	MODIFIED NAME
Button	
Button_1	
Button_2	
Button_3	Open Order - toolbar
Button_4	
Button_5	



What visual cue indicates that a GUI Map File has been changed but not saved?

6. Save the modified GUI File.

In the GUI Map Editor GUI File list, click lab11.gui to select it.
Select File > Save.

7. Close the GUI Map Editor.

Add New Objects to Existing GUI File



1. Start a new test in WinRunner.



2. Start recording.



3. Set the Flight application to its initial conditions.



4. Open order number 1.

Click the *Open Order* toolbar button.
Check the *Order No.* check box.
Type 1 in the *Order No.* edit box.
Click OK.



5. Stop recording.



Did the modified name of Button_3 appear in the script?

6. Display lab11.gui and the temporary buffer at the same time.

Open the GUI Map Editor and press the *Expand* button. The Editor expands with “L0 <Temporary>” displayed in both panes.
Select “L1 lab11.gui” from the *GUI File* drop-down list in the left pane.



What new windows / objects were learned during recording?

7. Move the Open Order window and its objects in <Temporary> to lab11.gui.

Select the window “Open Order” in the <Temporary> pane.

Press the *Move* button. A message window appears.

Click *OK* in the message window.



Did the Open Order window and all its objects move to the lab11.gui file?

8. Save the modified file, lab11.gui, then close the GUI Map Editor.



9. Save the WinRunner script as C:\test_project\tests\lab11a

Exercise C: Loading and Unloading GUI Map Files



What visual cue is displayed in the GUI Map Editor when a GUI Map file is loaded? What is the visual cue when a GUI Map file is open but not loaded?



What TSL statement can you use to unload and close all GUI Map files which may be open? Where should you place this statement in the script?

1. **Enable your script to unload and close all GUI Map files automatically before loading the current GUI Map file.**

At the top of your test script, add the following TSL statement:

```
GUI_close_all();
```



What TSL statement can you use to load a GUI Map file automatically from your script? Where should you place this statement in the script?

2. **Enable your script to automatically load lab11.gui.**

Insert a blank line after the `GUI_close_all` statement and add:

```
GUI_load("C:\\test_project\\GUI_Map_files\\lab11.gui");
```



NOTE: In WinRunner's TSL (Test Script Language), the backslash is a control character. To identify the path of a file in WinRunner, you must use two backslashes rather than one.



3. **Save the script.**

4. **Close all GUI Map files, then close the GUI Map Editor.**

Select File > Close All.

5. **Play back the script from the top in Debug mode to verify correct execution.**



How do you know that the GUI Map file was loaded properly?

Exercise D: Fixing a GUI Map File

Setup

1. **Open the test** `C:\merc_trn70\wr\Lab11_create_order`.
2. **Play back the test script from the top in Debug mode.**



Did the script replay without error? If not, what was the error?

3. **Press *Cancel* in the Run Wizard dialog.**

Using the GUI Spy

1. **View the description of the *Flights* button using the GUI Map Editor.**
Open the GUI Map Editor. Locate the object “FLIGHT” in the GUI Map file `Create_Trn.gui`.
2. **View the description of the *Flights* button using the GUI Spy.**
From the GUI Map Editor, select `Tools > GUI Spy`. The GUI Spy appears.
Select *Recorded Properties* in the *Show in description* frame.
Select the *Objects* radio button in the *Spy on* frame.
Click on the *Spy* button.
Place the cursor over the *Flights* button in the Flight application. The *Flights* button’s description appears in the *Description* field of the GUI Spy.
Press the `Left-Ctrl+F3` softkey to stop spying.



If `Left-Control+F3` doesn’t work, try `Left-Ctrl+3`.



NOTE: The GUI Spy can also be invoked using the WinRunner menu command `Tools > GUI Spy`.



In the boxes below, write the description of the *Flights* button as shown by the GUI Spy and by the GUI Map Editor.

DESCRIPTION - IN GUI SPY	DESCRIPTION - IN GUI MAP EDITOR



Explain how you would fix the GUI Map file.

3. Make the necessary corrections to the GUI Map file using the GUI Map Editor so that your test can play back properly.

4. Close the GUI Map Editor.

5. Put the execution arrow on the line: `button_press ("FLIGHT") ;`



6. Press the *Step* icon.

If the Flights Table opens, you fixed the GUI Map file correctly and you should save the change(s) you made to it.

7. Play back the script to verify correct execution.



Did the test play back successfully?

8. Close all scripts and GUI files.

Exercise E: Adding the GUI Map Editor to the Custom User Toolbar

In this exercise you will use the Custom Toolbar feature in WinRunner to give you quick access to important menu commands.

1. Open the Customize User Toolbar dialog box.

In WinRunner, select Settings > Customize User Toolbar.



In what category would the GUI Map Editor be found?

2. In the Customize User Toolbar dialog, select “Tools” from the *Category* list.



Which command refers to the GUI Map Editor?

3. Check the “Edit GUI Map” check box and press OK.



Was a GUI Map Editor icon added to your toolbar?

4. Invoke the GUI Map Editor from the icon toolbar.

5. Close the GUI Map Editor.

Review: Managing GUI Map Files

Test your understanding of this lesson by answering the following questions:



1. Assume you are about to begin creating automated tests for an application. Would you consider using one GUI Map file per test instead of one GUI Map file for the entire application? Please explain your answer.



2. When do you need to update a GUI Map file?



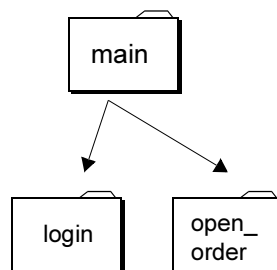
3. Select the correct statement.

When a new version of the application is released and has changes in the user interface:

- (a) A new GUI Map file should be created from scratch.
- (b) The GUI Map file from the previous version should be used without any changes.
- (c) The GUI map file from the previous version should be modified according to the changes in the application.

EXERCISES: Batch Testing

In this section you will create a simple modular test tree consisting of a main test which calls two other tests and passes parameter values.



- The **main** test will call the login and open_order tests, then exit the Flight application.
- The **login** test is already created. It invokes the Flight application and opens an order, then performs a checkpoint.
- The **open_order** test is also already created. It opens an order and performs another checkpoint.

The login test will receive the values for the agent name and password from the main test.

Finally, you will run these tests in unattended batch mode.

Exercise A: Parameterize the Login Test

1. Verify GUI Map Setup

Select Settings > General Options.

Click on the *Environment* tab.

The *Global GUI Map File* radio button should be selected.

Click the OK button to exit the General Options dialog.

Select Tools > GUI Map Editor.

The only GUI Map file loaded should be flight70.gui.



2. Open the login test.

Select File > Open.

In the *Open Test* window navigate to:

C:\merc_trn70\WR\batch\

Select and open the *login* test case.

3. Insert paramter names into the edit_set statements.

In the line which reads:

```
edit_set ("Agent Name:", "jojo");
```

Replace the name “jojo” with the parameter name: login_name

In the line which reads:

```
edit_set ("Password:", "topsecret");
```

Replace the password “topsecret” with the parameter name: login_pw

4. Add the parameters to the test.

Select File > Test Properties.

Click the *Parameters* tab of the Test Properties dialog.

Click the *Add* button.

Type login_name in the *Name* field of the Parameter Properties dialog.

Click the OK button to close the Parameter Properties dialog.

Click the *Add* button again.

Type login_pw in the *Name* field.

Click the OK button to close the Parameter Properties dialog.

The Test Properties window should now list login_name and login_pw in the *Test Parameters* box.

Click the OK button to close the Test Properties dialog.



NOTE: Changes made in the Test Properties dialog are automatically saved in a file called *header*. They are not saved in the *script* file.



5. Save the login test.

Exercise B: Create the Main Test

In this exercise you will create a main test that calls two subtests. The main test script should be saved to the same directory as the other test scripts, so to call the test scripts you only need the test's name and not the full path.



1. Open a new test in the WinRunner window.



What statement should be used to call the login test?

2. Insert a `call` statement for the login test.

Type in the statement that will call the login test.



How should we modify this `call` statement to pass your name and the password "mercury" to the login test as parameters?

3. Modify your `call` statement to pass the parameters.



What statement will call the `open_order` test?

4. Insert the statement to call the `open_order` test.

Type in the `call` statement at the end of the script.
Press Enter.

5. If the Flight 1A application isn't open, invoke it now.

From the Windows Start menu, select Programs > WinRunner > Sample Applications > Flight 1A.

6. Log in with any name and the password: mercury



7. Start Recording in WinRunner.

Press the *Record* toolbar button, or select Create > Record - Context Sensitive.

8. Close the Flight application.

Select File > Exit in the Flight application.

Two lines should appear in your script, a `set_window` and a `menu_select_item`.



9. Stop recording in WinRunner.

Press the *Stop* toolbar icon, or select Create > Stop Recording.



10. Save the test as: C:\merc_trn70\WR\batch\main

Exercise C: Run Batch Test

We have created a modular testing tree by creating a main test which calls two other tests. Now we will run the tests in Batch mode. In this mode, WinRunner does not pause to display error messages or to notify the user about mismatches. In addition, a separate set of results is created for each called test, just as if you had run each test separately.

1. Confirm Batch mode is not selected.

Select Settings > General Options.

Click on the *Run* tab.

The *Run in batch mode* radio button should NOT be checked.

Click the OK button.



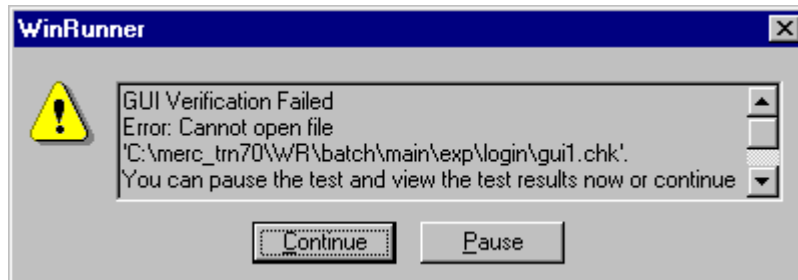
2. Run the main test.

Select the main test from WinRunner's Window menu.

Press the *Run from Top* button on the toolbar.

Click OK to accept the default name for the test results, Res1.

The following error message should appear:



3. Stop the test run.

Click *Pause* to clear the error message and stop the test run.



WinRunner cannot find a checkpoint's expected results. Why not?



There are two solutions to this problem. What are they?

4. Turn on Batch mode.

Select Settings > General Options.

Click on the *Run* tab.

Check the *Run in batch mode* check box.

Click the *OK* button.

5. Close the Flight application.



6. Run the main test.

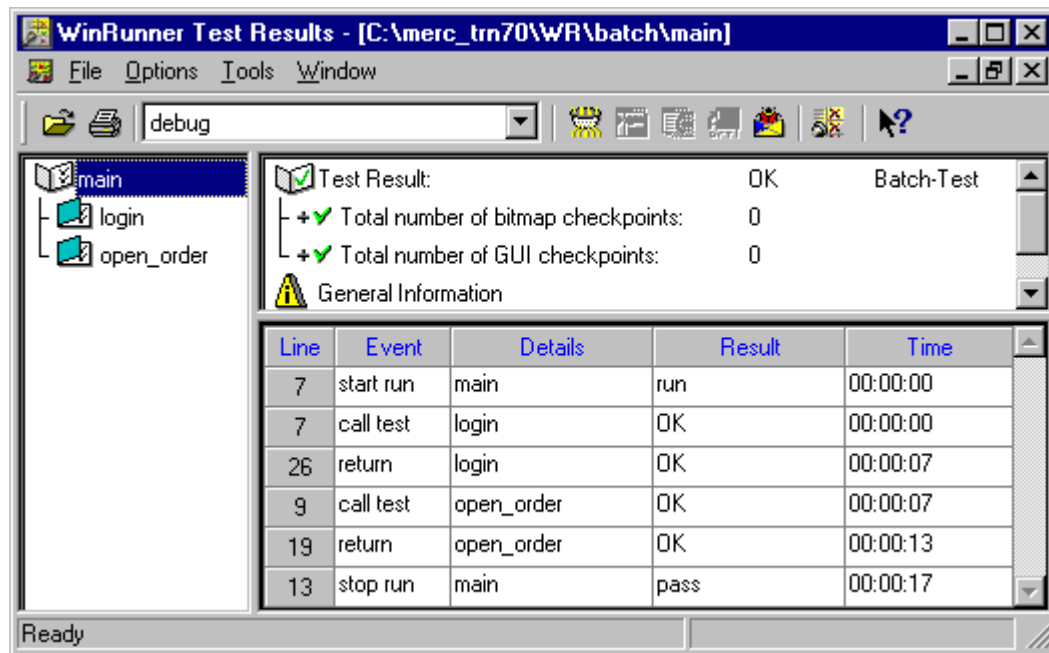
Select the main test from WinRunner's Window menu.

Press the *Run from Top* button on the toolbar.

Accept the default name for the test results, Res2.

7. Verify that the test completed successfully.

The Test Results window appears.



Verify that none of the tests reported errors.



NOTE: Click on the login and open_order subtests to view their results.

10

Summary and Practice Case



Preview

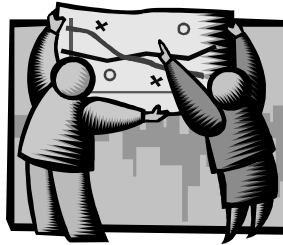
The practice case is an optional opportunity to put together all you have learned to this point.

Objectives

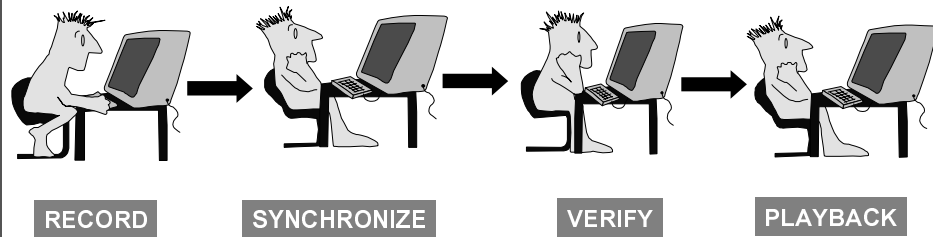
- summarize and review Lessons 1 through 9
- automate a complete test case from beginning to end

Plan the Testing Process

- Gather test documentation
 - what type of testing is required for the AUT?
 - which test data to use?
 - what results are expected?
- Learn the AUT
 - what screens to navigate?
 - what visual cues to establish?
- Confer with project team
 - functional experts
 - application experts



Key Terms in the Test Automation Process





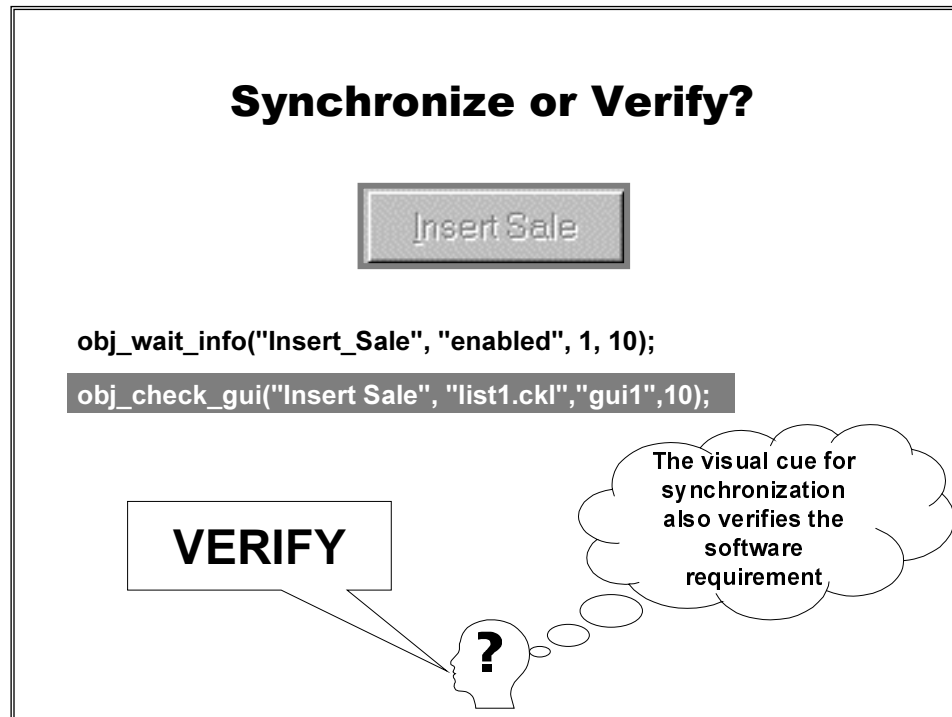
Record

Context Sensitive	Analog
When the application is based on GUI objects	When the application has non-GUI areas (Drawing application)
Generates more intuitive and maintainable script	When mouse tracks are necessary for correct execution
	When you are unable to use Context Sensitive mode



NOTE

For test case independence, set your test back to initial conditions during recording



When the visual cue for synchronization and verification are the same, it is recommended that a verification statement be used instead of the synchronization statement. In this example, the `obj_check_gui` statement contains a time parameter, which is the local time interval, in seconds, that WinRunner will wait to verify the actual GUI data.

Synchronize



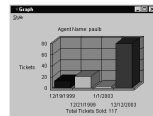
Wait for a window

```
set_window ("Flight Reservation", 20);
```



Wait for an object state/property value

```
obj_wait_info ("Bar", "label", "Insert Done...", 20);
```



Wait for a bitmap

```
obj_wait_bitmap ("Window", "Img1", 10, 209, 170, 81, 20);
```

Verify

What Do You Need To Verify?

Example:

Test Summary

Test Name:	Create_Order
Test Description:	This test verifies that a new flight reservation order is
Initial Conditions:	Flight Reservation System main window
Expected Results:	1) an order number is generated in the Order Number field 2) Insert Order button is disabled

Refer to test case documentation or software specifications and the AUT

SOFTWARE REQUIREMENT SPECIFICATIONS

2.3.2 Creating a New Order
Once the end user selects File-->New and enters their name, the "Flights" button is automatically enabled.

Verify

Which Method Should You Use?

GUI checkpoint

- Not affected by GUI changes in position or location
- Use to verify one or more properties of a single GUI object or multiple standard GUI objects

Example:

```
win_check_gui("Insert Sale","list1.ckl","gui1",1);
```

Verify

Which Method Should You Use?

Bitmap checkpoint

- Affected by GUI screen changes (fonts, colors, resolution)
- Use to check non-GUI areas of the application

Example:

```
obj_check_bitmap("ColGraph", "Img1", 1);
```

Verify

Which Method Should You Use?

Text verification

- Verify text from bitmaps or non-GUI objects
- Tester creates
 - expected result value
 - comparison of expected and actual values
 - comparison outcome information to Test Results

Example:

```
VersionNum = "Version 2.0";  
win_get_text ("About Flight", text, 43, 87, 107, 96);  
if (VersionNum == text)  
    tl_step ("Check Version", PASS, "The version is correct");  
else  
    tl_step ("Check Version", FAIL, "The version is incorrect");
```

Record & Playback Modes

Summary

Recording

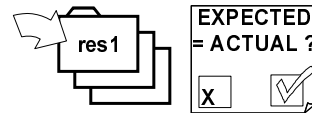


Results captured
in 'exp' as
EXPECTED



Playback
in 'Verify'
mode

Results captured
in 'res' as ACTUAL



Playback in
'Update' mode

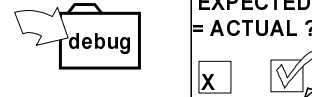


Results captured in
'exp' as new
EXPECTED



Playback in
'Debug'
mode

Results captured in
'debug' as ACTUAL



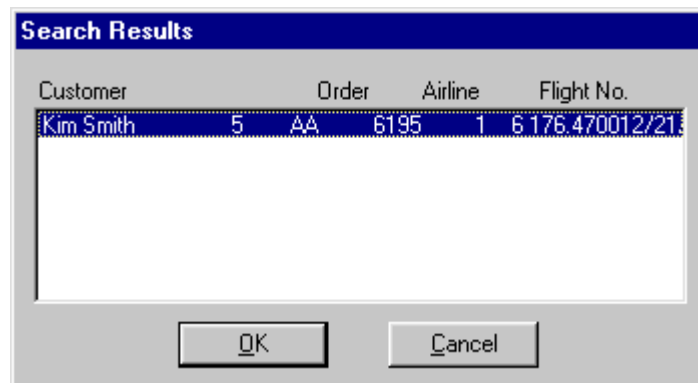
APPLICATION UPGRADE PROJECT TEST SPECIFICATION

Application: Flight Reservation System

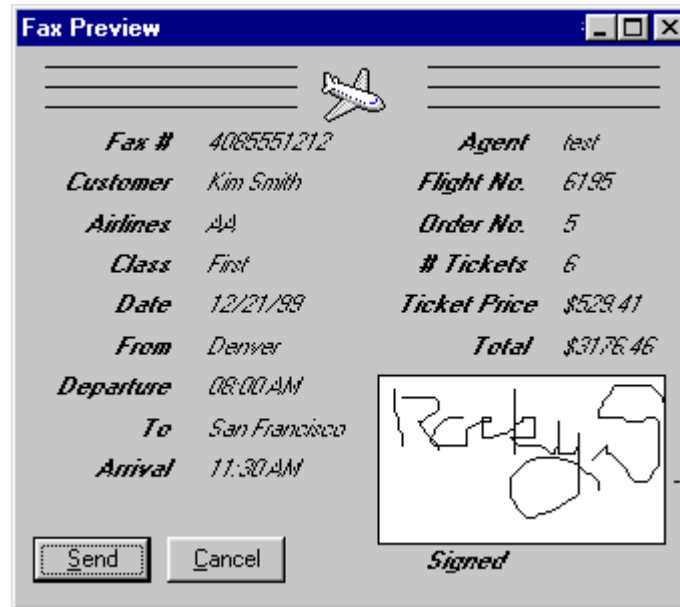
Business Process: Send Fax from Fax Preview

Description: Retrieve the order for customer Kim Smith; fax the order with agent's signature

Windows: Search Results Window



Fax Preview Window



The image shows a 'Fax Preview' window with a blue title bar. Inside, there are two columns of flight details. A small airplane icon is centered between the columns. At the bottom left are 'Send' and 'Cancel' buttons. At the bottom right is a signature box with a handwritten signature and the word 'Signed' below it.

Fax #	4065551212	Agent	test
Customer	Kim Smith	Flight No.	6195
Airlines	AA	Order No.	5
Class	First	# Tickets	6
Date	12/21/99	Ticket Price	\$529.41
From	Denver	Total	\$3176.46
Departure	08:00 AM		
To	San Francisco		
Arrival	11:30 AM		

Signed

Test Summary

Test Name:	FR-GUI
Test Description:	This test verifies: (1) the initial state of the Search Results window (GUI check) (2) order number (text check), and signature (bitmap check) in the Fax Preview window
Input Data - Open Order window:	"Kim Smith" in <i>Customer Name</i> field
Expected Results (1) - Search Results window:	(1) "Kim Smith" appears in the <i>Customer</i> list for all retrieved orders (2) <i>OK</i> and <i>Cancel</i> buttons are enabled
Expected Results (2) - Fax Preview window:	(1) Kim Smith's order appears in the window (2) Agent's signature appears in <i>Signed</i> box

INITIAL CONDITIONS OF TEST

- | |
|--|
| (1) The Flight Reservation Login window is visible |
| (2) All fields are clear of data |

User Actions

STEP	WINDOW TO RECEIVE INPUT	STEP DESCRIPTION	EXPECTED RESULTS
1	Login	Type your name in <i>Agent Name</i> field	
2		Type <i>mercury</i> in <i>Password</i> field	
3		Press <i>OK</i> button	Flight Reservation window appears
4	Flight Reservation	Press <i>Open Order</i> toolbar icon	Open Order window appears
5	Open Order	Check <i>Customer Name</i> check box	<i>Customer Name</i> edit field becomes active
6		Type <i>Kim Smith</i> in <i>Customer Name</i> edit field	<i>OK</i> button becomes active
7		Press <i>OK</i> button	The initial state is: (a) Search Results window appears (b) List includes "Kim Smith" as <i>Customer</i> (c) <i>OK</i> and <i>Cancel</i> buttons are enabled.
8	Search Results	Select the order for "Kim Smith"	
9		Press <i>OK</i> button	(a) Flight Reservation window appears (b) Kim Smith's order is displayed
10	Flight Reservation	Select File > Fax Order	Fax Order window appears

STEP	WINDOW TO RECEIVE INPUT	STEP DESCRIPTION	EXPECTED RESULTS
11	Fax Order	Type 4085551212 in <i>Fax Number</i> field	
12		Check <i>Send Signature</i> check box	
13		Sign name in <i>Agent Signature</i> field	
14		Press <i>Preview Fax</i> button	(a) Fax Preview window appears (b) Kim Smith's order is displayed (c) <i>Signed</i> box contains signature from previous window
15	Fax Preview	Press <i>Send</i> button	(a) Flight Reservation window appears (b) "Fax Sent Successfully" appears in progress bar.
16	Flight Reservation	Select File > Exit	Application shuts down

Application Upgrade Project - Test Specification

EXERCISES: Practice Case

You are the WinRunner expert for your organization's IS department. Your task is to plan and implement an automated test using the Application Upgrade Project Test Specification from the preceding pages. All the information you need to conduct the test is contained in the specification. To help you get started with planning, answer the following questions:



Refer to the *Introduction to Verification* lesson if you need help.



Which method of verification will you use to verify the initial state of the objects in the Search Results window?



Which method of verification will you use to verify that the Fax Preview window displays the correct order?



How will you ensure that the agent signature is put in exactly the same location each time the script is run?



Which method of verification will you use to verify that the Fax Preview window displays the agent signature?



What visual cue are you waiting for after pressing the *OK* button on the Login window?



How will you synchronize the test with the AUT after logging in?



How will you synchronize the test with the AUT after sending the fax?

1. Automate the test using Flight1A as a baseline.
2. Save the test as: C:\test_project\tests\lab10
3. After the test is working correctly, play it back on Flight1B.
Press *Continue* if any WinRunner message windows appear.



What defect(s) did you find in Flight1B as a result of this test?



Which attribute did you use to check that an order for customer Kim Smith appeared in the Search Results window?

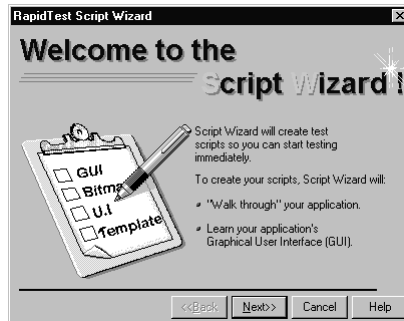
4. Close all tests when you have finished this exercise.

A

Appendix: Using the RapidTest Script Wizard

The RapidTest Script Wizard allows WinRunner to learn all windows and objects in an application in a series of steps. It systematically opens each window and learns all the objects it contains. Following a series of steps will lead you to save all information learned into a GUI Map file.

In this lesson, you will learn how to generate the GUI Map file for the Flight Reservation System application using the RapidTest Script Wizard.



Appendix A:

Using the RapidTest Script Wizard

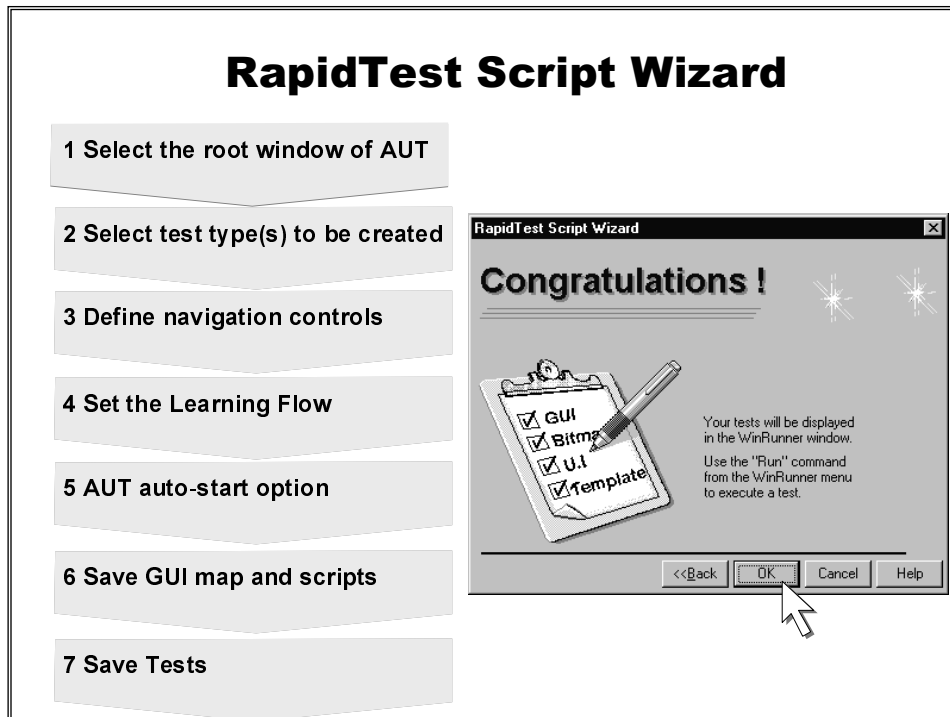
RapidTest Script Wizard

- "Walks" through application and learns all windows and objects
- Use Comprehensive mode
 - pauses after each window learned
 - allows you to modify logical names which appear in test scripts
 - allows you to map custom objects to standard objects, which makes test scripts context sensitive
- GUI Map File
 - the Wizard stores all information in a GUI Map file
 - for multiple testers, store GUI Map file on a shared network drive



NOTE

This is why you must create a comprehensive GUI Map before any test script generation



Exercise: Create the GUI Map using the RapidTest Script Wizard

In this exercise you will learn how WinRunner automatically creates the GUI map using the RapidTest Script Wizard. Until now, you used a GUI map file that was prepared for you. You will first close this file, so that you have an empty GUI Map.

Setup

- 1. Open the GUI Map Editor.**
Select Tools > GUI Map Editor.
- 2. Close all GUI map files.**
Select File > Close All in the GUI Map Editor.
If a dialog box appears asking you to confirm clearing the temporary buffer, click *Yes*.
- 3. Close the GUI Map Editor.**
Select File > Exit.
- 4. Open the Flight application 1a.**
Log in and display the Flight Reservation window.
- 5. If you are working on Windows/NT, change the Delay option to 5 seconds.**
For Windows NT only:
Choose Settings > General Options.
In the *Run* tab, modify the *Delay for window synchronization* option to 5000 milliseconds (5 seconds) and press *OK*. (This is done in order for the Test Wizard to wait longer for windows in the Flight application.)

RapidTest Script Wizard

- 1. Start the WinRunner RapidTest Script Wizard.**
Select Create > RapidTest Script Wizard.
Move the Wizard window so that the Flight Reservation window is visible.
Press *Next* in the Welcome window.

2. Identify your application.

Press the *Hand* button and click on the Flight Reservation window.
Press *Next*.

3. Deselect tests.

The Wizard can create automated tests. However, for the purpose of this exercise, you will use the Wizard only to create the GUI map.
Deselect all the tests.
Press *Next*.

4. Skip the Define Navigation Controls window.

Press *Next*.

5. Set the Learning Flow.

Select *Comprehensive*.
Press *Learn*.
Wait for WinRunner to learn the window.

6. Modify Logical Names.

Scroll down the list and select “Button_4”.
Identify the object that flashes in the Flight Reservation window.
Select “Button_4” again to see the object flashing.



What is the actual object in the application? Suggest a more descriptive name for this object.

7. Modify the logical name “Button_4” to “ToolBarNew”.

Select “Button_4” in the Objects list.
Press the *Modify* button.
Replace “Button_4” with “ToolBarNew” in the *Logical Name* field.
Press *OK*.

8. Identify the “ToolBarNew” object.

Select “ToolBarNew” in the *Objects* list and see the button flashing in the application.

Now when you record a test and select this button, WinRunner will use the name “ToolBarNew”.

9. Skip the Modify Logical Names window.

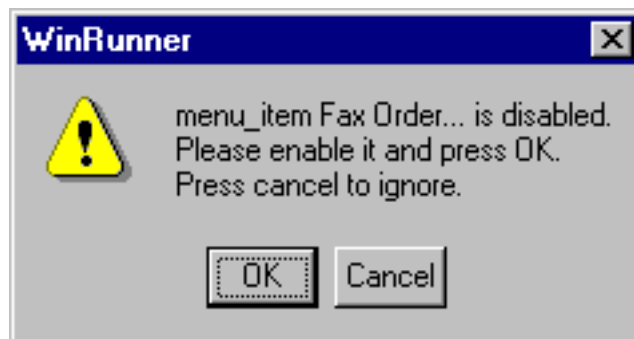
Select *Skip this screen from now on*.

Press *Next*.

Wait for WinRunner to learn all the other windows. The process takes several minutes.

10. Confirm the WinRunner message.

The learning process is paused when it reaches the Fax Order window in the Flight application. The following message appears:



11. Enable the Fax Order menu.

Select File > Open Order from the Flight Reservation window.

Select the *Order No.* checkbox, and type 1.

Press *OK* to open the order.

Press *OK* in the WinRunner Message box.

WinRunner learns the Fax Order window.

12. Wait for WinRunner to learn the rest of the windows.

Follow the instructions in the WinRunner messages displayed during the learning process.

If any more error alerts appear, press *Cancel* to ignore each of them.

13. Confirm the Start Application window.

The *No* radio button should be selected.

Press *Next*.

The Flight application will not be started automatically with WinRunner.

14. Save Files.

Save the Startup script under: `c:\merc_trn70\wr\startup`

Save the GUI Map file as: `c:\merc_trn70\wr\my_gui.gui`

Press *Next*.



Normally, when working in a testing group, you would want to store the Startup test and the GUI file on a network drive shared by all testers.

15. Complete the learning process.

Press *OK* in the Congratulations window.

The RapidTest Script Wizard created the GUI map.

16. If you are working on Windows/NT, change the Delay option back to 2 seconds.

For Windows NT only:

Choose Settings > General Options.

Modify the *Delay for window synchronization* option to 2000 milliseconds and press *OK*.